CS-600: Software Design and Development

5-2 Activity: Implementing a Framework-Based Web Interface

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 18, 2026

Framework Integration and UI Functionality

For this activity, I integrated the Vaadin framework into the existing Java application to create a simple web-based interface that connects the user interface to application data. The main interface uses a drawer-based side navigation panel, a header with a logo image, the application name Module5, and my name. I also created a BooksView page named Books List. This view uses a Vaadin Grid component to display book records from a TestData interface. The grid includes clear column headers for title, author, ISBN, year, and category, and it is set up as a read-only table, so users can review records without accidentally changing the data. This implementation proves the Read part of CRUD because the interface pulls placeholder book data from the back end TestData structure and displays it through the Vaadin part layer. Even though the data is not connected to a production database for this activity, the design keeps the UI separate from the data source. That separation is important because the TestData interface could later be replaced with a service or repository class without rewriting the whole view.

Modular Design, Maintainability, and User Experience

Using Vaadin supports modular design because each major responsibility is organized into a separate class or package. The MainLayout class controls the overall application shell, including the header, drawer, and side navigation. The BooksView class controls the books page. The Book class stands for one book record, and the TestData interface provides the placeholder data used by the view. This structure makes the application easier to understand because the layout, view, and data responsibilities are not mixed in one large file. The framework also improves

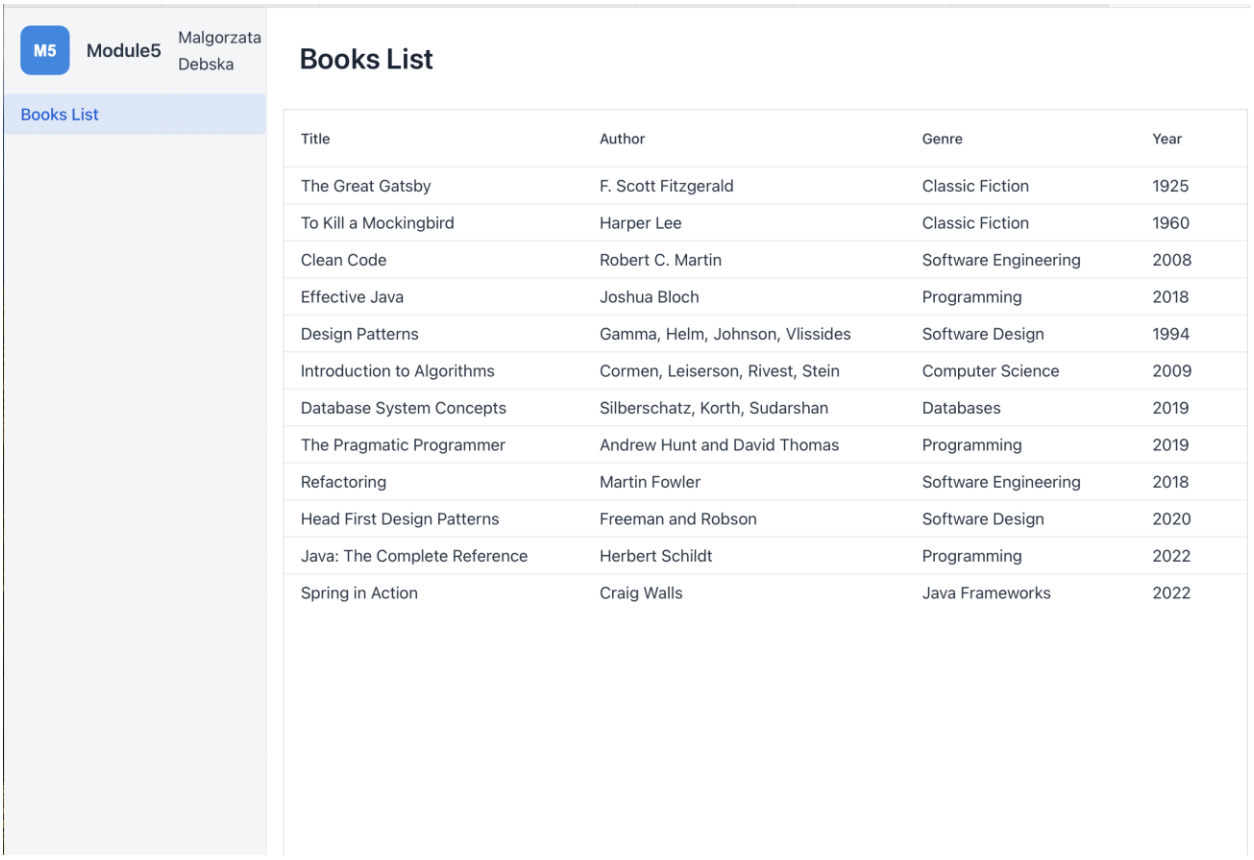
maintainability. Vaadin provides ready-made UI components, such as Grid, VerticalLayout, Image, and SideNav, which reduces the amount of custom front-end code needed. Because the interface is written in Java, the UI can work closely with existing Java logic and Spring Boot services. This makes future updates easier for a development team because developers can add new views, adjust navigation, or replace the placeholder to test data with database logic while keeping the same overall application structure. From a user experience perspective, the drawer navigation makes the interface easy to move through, while the header clearly shows the application and the person responsible for the activity. The Books List grid is also user-friendly because it presents information in labeled columns and supports scrolling when the list is longer than the visible page area. The read-only design keeps the activity simple and helps prevent unwanted edits while still allowing the user to verify that the UI is communicating with the data layer.

Testing, What Worked Well, and Challenges

To verify the work, I would run a Maven clean and compile, start the Spring Boot application, and open the Vaadin application in the browser. After the application loads, I would check that the drawer's navigation appears, that the header displays the logo image, Module5, and my name, and that the Books List option opens the BooksView page. I would also confirm that the grid displays all test records, that the column headers are visible, that the grid scrolls properly, and that the user cannot directly edit the grid records. One part that worked well was using Vaadin components to build the interface quickly without needing separate HTML, CSS, and JavaScript files. The Grid part was especially helpful because it automatically creates a structured table layout and allows the data to relate to only a few lines of Java code. The main challenge was making sure the implementation matched the specific activity requirements, especially changing

the header from the starter application style to the required Module5 layout and adding a separate BooksView that implements the TestData interface. Testing is important because small routing, menu, or package-name errors can prevent a Vaadin view from appearing correctly in the browser.

Screenshot of Working UI



Title	Author	Genre	Year
The Great Gatsby	F. Scott Fitzgerald	Classic Fiction	1925
To Kill a Mockingbird	Harper Lee	Classic Fiction	1960
Clean Code	Robert C. Martin	Software Engineering	2008
Effective Java	Joshua Bloch	Programming	2018
Design Patterns	Gamma, Helm, Johnson, Vlissides	Software Design	1994
Introduction to Algorithms	Cormen, Leiserson, Rivest, Stein	Computer Science	2009
Database System Concepts	Silberschatz, Korth, Sudarshan	Databases	2019
The Pragmatic Programmer	Andrew Hunt and David Thomas	Programming	2019
Refactoring	Martin Fowler	Software Engineering	2018
Head First Design Patterns	Freeman and Robson	Software Design	2020
Java: The Complete Reference	Herbert Schildt	Programming	2022
Spring in Action	Craig Walls	Java Frameworks	2022